Express.js Installation and Basic Setup –

1. Prerequisites for Express.js

Before working with Express.js, the following must already be installed on your system:

1. **Node.js**
    - Required because Express.js is installed and run using Node.js.
2. **Code Editor**
    - Example: **Visual Studio Code (VS Code)**

---

2. What is Express.js?

- **Express.js** is a **web framework for Node.js**.
- It provides pre-written libraries and modules that simplify backend development.
- It handles:
    - HTTP requests
    - Routing
    - Server creation
    - Middleware support

---

3. Installing Express.js Using npm

Step 1: Initialize a Node.js Project

Inside your project folder, open the terminal and run:

```
npm init -y
```

Explanation:

- `npm init` initializes a Node.js project.
- `-y` automatically answers all configuration questions with default values.
- This creates a `package.json` file, which is required before installing any npm packages.

Step 2: Install Express.js

Run the following command:

```
npm install express
```

(or shorthand)

```
npm i express
```

Result:

- Express.js and its dependencies are installed.
- A `node_modules` folder is created.
- A `package-lock.json` file is generated.
- Express appears under `dependencies` in `package.json`.

---

4. Project Structure After Installation

After installing Express.js, the project contains:

- **package.json**
  - Project configuration and dependencies
- **package-lock.json**
  - Detailed dependency tree with exact versions
- **node_modules/**
  - Contains Express.js and all required libraries

---

5. How Express.js Works (Core Concept)

1. A user sends an **HTTP request** (e.g., opening a webpage).
2. Express.js receives the request.
3. Express checks the request type (CRUD):
   - **Create**
   - **Read**
   - **Update**
   - **Delete**
4. Express processes the request and sends a **response** back to the user.

Example:

- Reading a page → `GET` request
- Adding data to database → `POST` request

6. Basic Express.js Server Setup

Step 1: Create Entry File

Create a file named:

```
index.js
```

Step 2: Import Express.js

```
const express = require('express');
```

Explanation:

- `require()` is used to import installed npm packages.
- This allows Express.js features to be used in the file.

Step 3: Create an Express App Object

```
const app = express();
```

- `app` represents the Express application.
- All Express methods are called using this object.

Step 4: Start the Server

```
app.listen(3000, () => {
  console.log('Successfully connected on port 3000');
});
```

Explanation:

- `listen()` starts the server.
- `3000` is the port number.
- The callback function runs once the server starts successfully.

> If port `3000` is already in use, you can use another port like `4000`, `5000`, etc.

---

7. Running the Server Using npm Scripts

Step 1: Add Script in `package.json`

```
"scripts": {
  "dev": "node index.js"
}
```

Step 2: Run the Server

```
npm run dev
```

- This starts the Express server.
- The console message confirms successful execution.

To stop the server:

```
Ctrl + C
```

---

8. Handling HTTP Requests (GET Route Example)

Example: Home Route

```
app.get('/', (req, res) => {
  res.send('Hello');
});
```

Explanation:

- `app.get()` handles **READ** requests.
- `/` represents the home route.
- `req` → Request object
- `res` → Response object
- `res.send()` sends data back to the browser.

Testing in Browser

Open:

```
http://localhost:3000
```

The browser will display:

```
Hello
```

---

9. Sending HTML as a Response

```
app.get('/', (req, res) => {
  res.send('<h1>Hello World</h1>');
});
```

- Express can send plain text or full HTML.
- Changes require restarting the server unless auto-restart is enabled.

---

10. Automatic Server Restart with Nodemon

Why Nodemon?

- Manually restarting the server after every change is inefficient.
- **Nodemon** automatically restarts the server on file changes.

Step 1: Install Nodemon

```
npm install nodemon
```

Step 2: Update npm Script

```
"scripts": {
  "dev": "nodemon index.js"
}
```

Step 3: Run Server with Nodemon

```
npm run dev
```

Benefits:

- No need to stop and restart the server manually.

- Server reloads automatically whenever files are saved.

---

11. Verifying Auto Reload

- Modify response text in `index.js`
- Save the file
- Refresh the browser
- Changes appear instantly without restarting the server

---

12. Summary of Key Steps

1. Install Node.js and VS Code
2. Initialize Node.js project (`npm init -y`)
3. Install Express.js
4. Create `index.js`
5. Import Express and create app object
6. Start server using `app.listen()`
7. Create routes using `app.get()`
8. Run server using npm scripts
9. Install and configure Nodemon for auto-restart
